

**Ministerul Educației și Cercetării al Republicii Moldova**

**Universitatea Tehnică a Moldovei**

**Facultatea Calculatoare, Informatică și Microelectronică**

**Departamentul Ingineria Software și Automatică**

# Laboratory Work 8:

The Same Calls Inside Your Own OS: Process Management and IPC in xv6

**Elaborated:** st. gr. FAF-243
Șoimu Ionuț

**Verified:** asist. univ.
Reițman Patricia

Chișinău – 2026

# Contents

**1 Lab 8: Track B – The Same Calls Inside Your Own OS (xv6) 2**
1.1 Before you start . . . 2
1.2 Part 1. run, inside xv6 . . . 3
1.2.1 Concept and Program Structure . . . 3
1.2.2 Execution and Experiments . . . 3
1.3 Part 2. pingpong . . . 6
1.3.1 Inter-Process Communication Across Unidirectional Pipes . . . 6
1.3.2 Implementation of user/pingpong.c . . . 6
1.3.3 Execution in xv6 . . . 7
1.4 Optional Extension: primes (Concurrent Prime Sieve) . . . 8
1.4.1 Algorithm and Process Architecture . . . 8
1.4.2 Implementation Details in xv6 . . . 8
1.4.3 Verification in QEMU . . . 9

**2 Conclusion 10**

# 1 Lab 8: Track B – The Same Calls Inside Your Own OS (xv6)

## 1.1 Before you start

### Purpose and Scope

In Laboratory Work 7, we investigated process creation, binary image replacement, and stream redirection on a host Linux system using the GNU C Library (`glibc`) and the Linux 6.x kernel. The primary objective of Laboratory Work 8 is to bridge user-space software abstractions with low-level kernel internals by implementing and evaluating the **exact same process management and Inter-Process Communication (IPC) primitives** inside **xv6-riscv**, a modern re-implementation of Dennis Ritchie and Ken Thompson's Unix Version 6 targeting 64-bit RISC-V hardware.

Unlike Linux user-space programs that rely on thousands of pages of standard library scaffolding, an xv6 user program executes directly on top of the bare-metal kernel interface. There is no `glibc`, no dynamic linker, and no complex runtime: every invocation of `fork()`, `exec()`, `wait()`, `pipe()`, `read()`, and `write()` transitions via RISC-V `ecall` instructions directly into the xv6 kernel trap handler.

### xv6 Workspace and Build Integration

Work is conducted in the repository tree `~/xv6-riscv`. In xv6, user-space utilities are compiled as standalone ELF binaries from `user/NAME.c` and embedded into the initial file system image (`fs.img`) created by `mkfs`. Adding a new program requires inserting `$U/_NAME\` into the `UPROGS` definition within the `Makefile`:

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_forktest\
    $U/_grep\
    $U/_init\
    $U/_kill\
    $U/_ln\
    $U/_ls\
    $U/_mkdir\
    $U/_rm\
    $U/_sh\
    $U/_stressfs\
    $U/_usertests\
    $U/_grind\
    $U/_wc\
    $U/_zombie\
    $U/_run\
    $U/_pingpong\
    $U/_primes\
```

Every target line begins with a leading tab character and concludes with a line-continuation backslash (\). We verified the compilation and execution of xv6 using QEMU:

```
➜  xv6-riscv make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
    nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,
    format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
```

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

## 1.2 Part 1. run, inside xv6

### 1.2.1 Concept and Program Structure

In Unix-like systems, the command-line shell executes commands through a classic three-step sequence:

1. **Process Spawning** (`fork`)**:** The shell duplicates its own address space, producing an identical child process.
2. **Program Invocation** (`exec`)**:** The child process executes `exec()`, replacing its virtual memory image with the targeted binary.
3. **Process Synchronization** (`wait`)**:** The parent suspends execution until the child completes, retrieving its termination status.

In xv6, these steps are executed through the minimal headers `kernel/types.h` and `user/user.h`. We implemented `user/run.c`:

```c
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6:  echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();                      // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }
  if(pid == 0){                          // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);                  // step 2: become echo
    printf("exec failed\n");             // runs only if exec failed
    exit(1);
  }
  int status;
  wait(&status);                         // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

### 1.2.2 Execution and Experiments

1. **Default Execution with** `echo`**:**

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

The parent process (PID 3) invokes `fork()`, allocating a newly initialized `proc` struct with PID 4. The child prints its PID, transforms into `echo` via `exec("echo", args)`, and outputs the argument string. When `echo` finishes, the parent's `wait(&status)` wakes up, reporting exit status 0.

2. **Modifying the Child to Execute `ls`:** We modified the argument array and `exec` invocation:

```
    char *args[] = { "ls", 0 };
    exec("ls", args);                // step 2: become ls
```

Rebuilding and launching `run` in QEMU produced:

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35864
echo           2 4 34760
sleep          2 5 34440
args           2 6 34424
mystat         2 7 35616
find           2 8 42048
run            2 9 35024
pingpong       2 10 35800
primes         2 11 36464
forktest       2 12 17704
grep           2 13 43288
init           2 14 35112
kill           2 15 34664
ln             2 16 34456
ls             2 17 41944
mkdir          2 18 34712
rm             2 19 34696
sh             2 20 57072
stressfs       2 21 35536
usertests      2 22 201520
grind          2 23 50336
wc             2 24 36832
zombie         2 25 34008
logstress      2 26 36680
forphan        2 27 35472
dorphan        2 28 34936
sync           2 29 34112
console        3 30 0
parent: child 4 ended with status 0
```

3. **Executing a Non-Existent Binary (`nosuchprog`):** We tested error recovery by replacing the executable target with an invalid name:

```
    char *args[] = { "nosuchprog", 0 };
    exec("nosuchprog", args);
```

Executing in QEMU:

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

**Observe:**

1. **Write the three calls that run.c uses in xv6 and the three calls run.c used on Linux in Lab 7. What is the same?**

   **xv6 calls:**

   - `fork()`
   - `exec(path, argv)`
   - `wait(&status)`

   **Linux calls (Lab 7):**

   - `fork()`
   - `execlp("ls", "ls", "-l", (char *)NULL)`
   - `waitpid(pid, &status, 0)` (or `wait(&status)`)

   **What is the same:**
   The high-level process execution model and interface are fundamentally identical. In both operating systems:

   - `fork()` separates creation from execution: it clones the parent's address space, returning 0 in the child and the child's PID in the parent.
   - `exec()` loads an ELF binary from storage, discards the old user memory pages, initializes new page tables, and begins execution at the entry point without altering the PID.
   - `wait()` puts the parent to sleep until a child terminates, recovering the child's exit status and allowing the kernel to reclaim its process table entry.

2. **Who prints "hello from the child": the code of run, or echo?**

   The program `echo` prints the message. Once `exec("echo", args)` succeeds, the code, data, and stack segments of `run.c` in the child process no longer exist in memory. The CPU registers and program counter are pointed to the entry address of the `echo` binary. The printed string is produced by the `printf()` call inside `echo.c`'s `main()` routine.

3. **Why is the status 1 here and 127 on Linux? (Look at the exit line after exec in both programs.)**

   The exit code received by the parent process is determined entirely by the integer parameter passed to `exit()` by the child when `exec` fails:

   - In xv6 (`user/run.c`):

```
    printf("exec failed\n");
    exit(1);
```

The child explicitly terminates with argument `1`. The xv6 kernel stores this value in `p->xstate`, and `wait(&status)` copies `p->xstate` into the parent's status variable.

- In Linux (`run.c`):

```
    perror("exec");
    exit(127);
```

The child explicitly terminates with argument `127`, following the standard POSIX shell convention where exit code 127 indicates that a specified command or binary was not found.

## 1.3 Part 2. pingpong

### 1.3.1 Inter-Process Communication Across Unidirectional Pipes

A pipe is an anonymous byte stream provided by the operating system kernel. In xv6, calling `pipe(p)` allocates a circular buffer in kernel memory and returns two file descriptors:

- `p[0]`: the read descriptor.
- `p[1]`: the write descriptor.

Because pipes in xv6 are strictly unidirectional, two distinct pipes are necessary to establish bidirectional communication between parent and child without race conditions:

1. `p2c` (Parent-to-Child): parent writes to `p2c[1]`; child reads from `p2c[0]`.
2. `c2p` (Child-to-Parent): child writes to `c2p[1]`; parent reads from `c2p[0]`.

### 1.3.2 Implementation of user/pingpong.c

We implemented `user/pingpong.c`:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
  char b = 'x';

  if(pipe(p2c) < 0 || pipe(c2p) < 0){
    fprintf(2, "pipe failed\n");
    exit(1);
  }

  int pid = fork();
  if(pid < 0){
    fprintf(2, "fork failed\n");
    exit(1);
  }

  if(pid == 0){
```

```
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    if(read(p2c[0], &b, 1) != 1){
      fprintf(2, "child read failed\n");
      exit(1);
    }
    printf("%d: received ping\n", getpid());
    if(write(c2p[1], &b, 1) != 1){
      fprintf(2, "child write failed\n");
      exit(1);
    }
    close(p2c[0]);
    close(c2p[1]);
    exit(0);
  }

  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  if(write(p2c[1], &b, 1) != 1){
    fprintf(2, "parent write failed\n");
    exit(1);
  }
  if(read(c2p[0], &b, 1) != 1){
    fprintf(2, "parent read failed\n");
    exit(1);
  }
  printf("%d: received pong\n", getpid());
  close(p2c[1]);
  close(c2p[0]);
  wait(0);
  exit(0);
}
```

### 1.3.3 Execution in xv6

We compiled xv6 with `pingpong` included in `UPROGS` and tested it in QEMU:

```
$ pingpong
6: received ping
5: received pong
$
```

The output order confirms proper synchronization: the child (PID 6) prints `received ping` first, followed by the parent (PID 5) printing `received pong`.

**Observe:**

1. **Why do you need TWO pipes, and not one for both directions?**
   A Unix/xv6 pipe is a single unidirectional FIFO queue. If a single pipe `p[2]` were shared for both directions:

   - When the parent writes a byte into `p[1]` and immediately calls `read(p[0])`, the parent could read back its own byte before the child process is scheduled.
   - If both processes read simultaneously from the same pipe buffer, non-deterministic race conditions occur.

   Allocating two separate pipes establishes two disjoint kernel buffers, isolating traffic from parent-to-child (`p2c`) from traffic in the return direction (`c2p`).

2. **Nothing in your code says ”wait until the child has replied”. Which call makes the parent wait?**

   The blocking system call `read(c2p[0], &b, 1)` forces the parent to wait. In xv6, reading from an empty pipe puts the process into the `SLEEPING` state on the pipe's internal condition channel (`piperead()`). The parent cannot proceed until the child writes the byte into `c2p[1]`, triggering a kernel `wakeup()` on the channel.

## 1.4 Optional Extension: primes (Concurrent Prime Sieve)

### 1.4.1 Algorithm and Process Architecture

The Prime Sieve pipeline (originally conceived by Doug McIlroy, inventor of Unix pipes) generates prime numbers using concurrent processes arranged as a pipeline:

- The root process generates the sequence of integers from 2 to 35 and streams them into a pipe.
- Each filter stage reads the first integer from its input pipe; by mathematical induction, this integer $p$ is guaranteed to be prime.
- The stage prints `prime p`, creates a new child stage with an outgoing pipe, and filters subsequent integers $n$, forwarding only those where $n \pmod{p} \neq 0$.

```
[Root 2..35] ---> [Stage 2] ---> [Stage 3] ---> [Stage 5] ---> ...
```

Figure 1: Process pipeline for the concurrent Sieve of Eratosthenes.

### 1.4.2 Implementation Details in xv6

In xv6, processes have a strict limit on open file descriptors (`NOFILE = 16`). Furthermore, recursive child spawning in bare-metal compilers like modern GCC 16 emits infinite-recursion warnings if not carefully managed.

To guarantee zero warnings and deterministic descriptor management, we implemented `user/primes.c` using an iterative stage handoff loop:

```c
#include "kernel/types.h"
#include "user/user.h"

void
sieve(int pleft[2])
{
  while(1){
    close(pleft[1]);
    int prime;
    if(read(pleft[0], &prime, sizeof(prime)) != sizeof(prime)){
      close(pleft[0]);
      exit(0);
    }
    printf("prime %d\n", prime);

    int pright[2];
    if(pipe(pright) < 0){
      fprintf(2, "pipe failed\n");
      exit(1);
    }

```

```
    int pid = fork();
    if(pid < 0){
      fprintf(2, "fork failed\n");
      exit(1);
    }
    if(pid == 0){
      close(pleft[0]);
      pleft[0] = pright[0];
      pleft[1] = pright[1];
      continue;
    } else {
      close(pright[0]);
      int n;
      while(read(pleft[0], &n, sizeof(n)) == sizeof(n)){
        if(n % prime != 0){
          write(pright[1], &n, sizeof(n));
        }
      }
      close(pleft[0]);
      close(pright[1]);
      wait(0);
      exit(0);
    }
  }
}

int
main(int argc, char *argv[])
{
  int p[2];
  if(pipe(p) < 0){
    fprintf(2, "pipe failed\n");
    exit(1);
  }

  int pid = fork();
  if(pid < 0){
    fprintf(2, "fork failed\n");
    exit(1);
  }
  if(pid == 0){
    sieve(p);
  } else {
    close(p[0]);
    for(int i = 2; i <= 35; i++){
      write(p[1], &i, sizeof(i));
    }
    close(p[1]);
    wait(0);
  }
  exit(0);
}
```

### 1.4.3 Verification in QEMU

Running `primes` inside xv6 produced all prime numbers between 2 and 35:

```
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
$
```

Every prime was accurately identified, and all stages exited cleanly after encountering EOF on their respective pipe descriptors.

## 2 Conclusion

In this laboratory work, I evaluated process management and inter-process communication directly inside **xv6-riscv**, verifying the fundamental principles explored on Linux in Laboratory 7.

Implementing `user/run.c` demonstrated that the core Unix system calls—`fork()`, `exec()`, and `wait()`—operate under the exact same semantic model in a lightweight educational kernel as they do on a full-scale Linux system. We confirmed that `fork()` clones the caller's process state, `exec()` discards the previous memory space to execute the target ELF binary, and `wait()` synchronizes parent execution with child termination while reaping process state.

Developing `user/pingpong.c` highlighted the mechanics of pipe-based IPC. Because pipes are strictly unidirectional, two distinct pipes are mandatory to avoid race conditions and self-reads. Furthermore, the blocking nature of `read()` on empty pipes naturally provides process synchronization without requiring explicit sleep loops or busy-waiting.

Finally, implementing the concurrent prime sieve in `user/primes.c` demonstrated the power and elegance of Unix pipelines, where complex algorithmic problems decompose into a chain of autonomous filtering processes communicating through kernel FIFOs. This laboratory solidified my understanding of how low-level system call primitives compose to provide robust process isolation and coordination.